

---

Multidisciplinary Project Proposal Report

---

# **SMART CLOSET**

By:

**Fatima Al Rabih  
Karen El Khoury  
Lea Chamseddine  
Marilyn Abou Tayeh**

Supervisor:

**Hanan Farhat**

May 20, 2025

Ecole Supérieure d'Ingénieurs de Beyrouth  
Université Saint-Joseph de Beyrouth

## Abstract

A personal wardrobe needs clothing organization, selection of outfits based on several factors, and tracking their wear, making it a complex task to manage. Traditional methods are ineffective and time-consuming, based on the user's memory resulting in unused clothing and uncoordinated outfits. The problem-solving goal is to create and develop a smart closet system using computer vision, artificial intelligence, and mobile technologies for automating wardrobe management.

The solution includes three components: (1) Clothes items put into or taken out of the closet through a tablet camera will be detected and recognized by YOLOv8 within a computer vision system, (2) A recommendation system that builds up clothes combinations according to color, weather, and user's events, in order to provide the most appropriate recommendations and (3) A mobile application that explains clothing use pattern and allows interaction with the closet system through a user-friendly interface.

Key achievements include the development of a detection model using YOLOv8 on clothing datasets trained through Roboflow, enabling considerable classification of objects inserted into or removed from the closet. The application is built with Android Studio and interacts with a PostgreSQL database through Supabase, enabling secure and efficient data handling. The outfit suggestions are generated taking into consideration the weather, occasion, and color coordination, with plans to add a learning algorithm that adapts over time to suit specific tastes of individual users.

The mobile application offers a smooth user experience to the users by allowing them to manage their wardrobe, see outfit history, and get outfit recommendations. The key milestones achieved are the successful development of the system architecture design, database schema development, model training for initiating clothing recognition, and deployment of key application features.

Future assignments include optimizing the AI model, enhancing recognition precision and refining suggestion algorithms, optimizing outfit suggestion by learning the user's taste, enhancing application features with real-time weather and calendar tracking, developing a fully independent backend, and optimizing database storage.

This project showcases how a multidisciplinary engineering approach combining artificial intelligence, data analytics, and software development can significantly enhance everyday tasks such as wardrobe management, improving efficiency, personalization, and user satisfaction.

## Acknowledgements

We, the members of Smart Closet, extend our deepest gratitude to everyone who contributed to the success of this project. This journey has been one of collaboration, learning, and mutual support, culminating in a project that we are proud to present.

First and foremost, we express our sincere thanks to our project supervisor, Dr. Hanan Farhat whose guidance, expertise, and patience were instrumental in steering this project towards its completion. Her suggestions and comments were of immense value, and her motivation challenged us to give our best.

We are also extremely grateful to the faculty and staff of Saint Joseph University (USJ), particularly to Dr. Juliana Rayes, supervisor of multidisciplinary projects in the Department of Mechanical and Electrical Engineering. We are grateful for her excellent supervision, for organizing and coordinating the projects, and for providing us with time and the opportunity to participate in such a fulfilling and enriching experience.

Special appreciation is given to our classmates, as well as Dr. Tina Yaacoub and Dr. Maria Habib who participated in our test phases of the application. Their time, feedback, and willingness to try our application added meaningful depth and authenticity to our work.

Lastly, we extend our gratitude to each other, the members of Smart Closet. This project was the result of a collaborative effort that required dedication, compromise, and teamwork. Through this experience, we have grown both individually and collectively, gaining valuable knowledge in addition to forming friendships that we deeply cherish.

This project report is not only a reflection of our hard work but also a testament to the support and guidance we received from all those mentioned above. Thank you for making this journey memorable and our project a success.

## Contents

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Abstract .....                                                                            | 2  |
| Acknowledgements .....                                                                    | 3  |
| List of Figures.....                                                                      | 7  |
| List of Tables .....                                                                      | 8  |
| 1 Introduction.....                                                                       | 9  |
| 2 Problem requirements and constraints .....                                              | 10 |
| 2.1 Functional requirements .....                                                         | 10 |
| 2.2 Constraints.....                                                                      | 10 |
| 2.2.1 Technical constraints.....                                                          | 10 |
| 2.2.2 Non-technical constraints .....                                                     | 10 |
| 2.2.3 Standards / codes / regulations / policies.....                                     | 11 |
| 3 Background / Research / Related work / Literature review.....                           | 11 |
| 3.1 History of Smart Wardrobe Management Systems: .....                                   | 11 |
| 3.2 Current Researchers in the Field.....                                                 | 11 |
| 3.3 State-of-the-Art Technologies in Smart Closet Systems .....                           | 11 |
| 3.4 Global Evaluation of Existing Solutions: Pros, Cons, and Methodology Differences...12 |    |
| 3.4.1 Samsung Family Hub™ refrigerators.....                                              | 13 |
| 3.4.2 Lucy.....                                                                           | 14 |
| 3.4.3 ACloset .....                                                                       | 14 |
| 4 Solution design.....                                                                    | 14 |
| 4.1 Multiple design alternatives comparison.....                                          | 14 |
| 4.1.1 RFID Tracking Clothes .....                                                         | 14 |
| 4.1.2 Manual Input on a Mobile application.....                                           | 15 |
| 4.1.3 Camera Based Clothes tracking with AI recognition.....                              | 16 |
| 4.2 Problem formulation.....                                                              | 16 |
| 4.3 Required resources .....                                                              | 17 |
| 4.3.1 Software Resources : .....                                                          | 17 |
| 4.3.2 Hardware Resources : .....                                                          | 17 |
| 4.4 Conceptual model.....                                                                 | 17 |
| 5 Project Management .....                                                                | 19 |
| 5.1 Planning.....                                                                         | 19 |
| 5.2 Project Facts and Figures.....                                                        | 20 |
| 5.3 Conflict management.....                                                              | 20 |
| 6 Solution Implementation.....                                                            | 20 |
| 6.1 Database Management .....                                                             | 20 |

|        |                                                                              |    |
|--------|------------------------------------------------------------------------------|----|
| 6.1.1  | Table Conception: .....                                                      | 20 |
| 6.1.2  | Security and Authentication:.....                                            | 21 |
| 6.2    | Mobile Application Backend.....                                              | 22 |
| 6.2.1  | Supabase: .....                                                              | 22 |
| 6.2.2  | FastAPI External Server .....                                                | 26 |
| 6.3    | User Interface Implementation .....                                          | 27 |
| 6.3.1  | User Interface Structure .....                                               | 27 |
| 6.3.2  | Tools and Techniques used .....                                              | 28 |
| 6.4    | IOT Automation.....                                                          | 28 |
| 6.4.1  | Mechanism .....                                                              | 28 |
| 6.4.2  | Hardware .....                                                               | 29 |
| 6.5    | Artificial Intelligence Implementation .....                                 | 29 |
| 6.5.1  | System Architecture .....                                                    | 29 |
| 6.5.2  | Image Pre-Processing Techniques .....                                        | 30 |
| 6.5.3  | Visual Similarity Detection Techniques .....                                 | 30 |
| 6.5.4  | Object Detection Model .....                                                 | 31 |
| 6.5.5  | Color Classification Module .....                                            | 35 |
| 6.6    | Testing, Evaluation and Iterative improvements.....                          | 36 |
| 6.6.1  | Testing Strategy .....                                                       | 36 |
| 6.6.2  | Questionnaire Results .....                                                  | 37 |
| 6.6.3  | Functional Testing .....                                                     | 38 |
| 6.6.4  | Iterative Improvements .....                                                 | 39 |
| 7      | Impact of the proposed solution .....                                        | 39 |
| 7.1    | Global and Local Impact .....                                                | 39 |
| 7.2    | Economic Impact .....                                                        | 40 |
| 7.3    | Environmental Impact .....                                                   | 40 |
| 7.4    | Impacted Sectors and Communities.....                                        | 40 |
| 8      | Conclusion.....                                                              | 40 |
| 8.1    | Key Milestones: .....                                                        | 40 |
| 8.2    | Future Work: .....                                                           | 41 |
| 9      | References .....                                                             | 42 |
| 10     | Appendix 1 .....                                                             | 43 |
| 10.1   | Supabase-Kotlin, query to get all available items in a specific closet ..... | 43 |
| 10.2   | Supabase Client Initialisation In Kotlin: .....                              | 43 |
| 10.3   | FastAPI Endpoint.....                                                        | 43 |
| 10.4   | Launching the Server.....                                                    | 43 |
| 10.4.1 | Uvicorn Local Server.....                                                    | 43 |

|      |                                                       |    |
|------|-------------------------------------------------------|----|
| 10.5 | Arduino RGB circuit .....                             | 43 |
| 10.6 | Accessing the Main-Fashion Dataset via Roboflow ..... | 44 |
| 10.7 | Color Extraction and Analysis Functions: .....        | 44 |

## List of Figures

|                                                                                                                                        |    |
|----------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1:Closet Management System:Conceptual Model 1 .....                                                                             | 18 |
| Figure 2:Closet Management System:Conceptual Model 2 .....                                                                             | 18 |
| Figure 3:Complete Functionality Map - UML diagram .....                                                                                | 19 |
| Figure 4:Gantt chart (on Microsoft Project).....                                                                                       | 19 |
| Figure 5:User flow for adding a new closet and clothing items, with photo upload and categorized display in the digital wardrobe. .... | 24 |
| Figure 6:Analytics and recommendation flow showing outfit history, frequently worn items, and personalized outfit suggestions.....     | 25 |
| Figure 7:Profile UI illustrating theme switch from pink to blue based on user preference.....                                          | 26 |
| Figure 8:YAML configuration file for a fashion object detection dataset, defining class labels and dataset structure. ....             | 32 |
| Figure 9:YOLO model training configuration with dataset and hyperparameters.....                                                       | 33 |
| Figure 10:Accuracy Metrics for YOLOv8 (Section 6.5.4).....                                                                             | 34 |
| Figure 11:Confusion Matrix .....                                                                                                       | 34 |
| Figure 12:User Experience Rating Distribution.....                                                                                     | 37 |
| Figure 13:Response Time Satisfaction Distribution.....                                                                                 | 38 |
| Figure 14:Feature Usability Comprehension Metrics.....                                                                                 | 38 |

## List of Tables

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Table 1:Comparative Analysis of Existing Smart Systems .....                    | 13 |
| Table 2:RFID-Based Clothing Tracking:System Overview .....                      | 15 |
| Table 3:Manual Clothing Inventory through Mobile app: System Overview .....     | 15 |
| Table 4:AI-Powered Camera-Based Clothing Tracking:System Overview.....          | 16 |
| Table 5:Project Team Composition Member Roles and Responsibilities .....        | 20 |
| Table 6:Applications Workflow: User Interactions and System Responses .....     | 23 |
| Table 7: Key Components and Tools for Clothing Detection and Segmentation ..... | 30 |
| Table 8:Test Environment Specifications.....                                    | 36 |
| Table 9:Overall Satisfaction & Usability Scores.....                            | 37 |
| Table 10:Version History: Iterative Development and Feature Enhancements.....   | 39 |



# 1 Introduction

Managing a wardrobe can be a complex challenging task. From organizing clothing, planning outfits and tracking clothing usage are a part of each individual everyday life.

Traditional wardrobe management is a redundant job for one person: Manual sorting, memory-based outfit selection, inefficient storage usage leads to wasted time planning for an outfit, forgotten clothing pieces that are being underused and non-optimal outfit choices depending on the occasion, trends and/or the weather. It is clear that quickly; by adding factors, wardrobe management becomes a complex engineering problem.

## **Problem identification:**

This project addresses a complex engineering problem that:

- Involves wide-ranging technical issues such as Clothing recognition (computer vision), recommendation systems, and mobile application development.
- Involves diverse stakeholders, including regular people/clients, mobile developers, AI engineers, Data Engineers, fashion specialist.
- Contains multiple component parts, including image processing, suggestion algorithms, mobile application integration, data processing and analysis.
- Requires a multidisciplinary approach: computer vision, machine learning, and data analytics.
- Has significant consequences in the optimality of outfits choice according to the event, or weather, in time efficiency, and in clothing usage efficiency.

## **Project objectives and outcomes:**

The Outcome of the project is to design and implement a smart closet system that has the following specifications:

1. Tracking inventory using Camera Detection: Using Computer Vision, and inventory will be put to the access of the user in order to have a better look on the content of the closet.
3. Outfit Suggestion: Based on several factors such as events, weather, user preference, and available clothing items.
4. Mobile application Integration: A smooth user interface where the user can interact with the mobile application where he/she can find all clothing items available in the closet, or request an outfit suggestion.
4. Data Analysis for Clothing usage: The mobile application displays the top three most frequently worn clothing items, items logged on the calendar, recently used garments, and forgotten items.

## **Expected Outcome:**

- Small Wooden Closet equipped with a tablet
- Mobile application on the tablet
- Automation and live tracking of the inventory using IOT Arduino board
- Recommendation system providing optimized outfit choices based on user preference, weather, occasion, and color.

**Report outline:**

Building upon the introduction, the problem requirements and constraints will first be defined, specifying both functional and technical limitations that guide the system's development. Next, existing smart wardrobe solutions and ongoing advancements in the field will be examined, with an assessment of their methodologies, strengths, and limitations to provide a comparative analysis. Subsequently, the solution design will be elaborated by exploring multiple design alternatives, evaluating their respective advantages and drawbacks, and justifying the final selection. This section will also include a formal problem formulation and an overview of the required software and hardware resources. The project management framework will then be discussed, covering planning, task distribution, and conflict resolution strategies to ensure effective collaboration among team members. Finally, the report will conclude with a comprehensive summary of the research and development progress, highlighting key insights and future directions.

## **2 Problem requirements and constraints**

### **2.1 Functional requirements**

The functional requirements define the key features the system must provide to fill its intended purpose.

- Closet prototype
- Offer a user-friendly mobile application for wardrobe management.
- Use a camera to detect and classify clothing items entering and exiting the closet.
- Detection model to classify and extract the image from the background.
- Provide outfit suggestions based on user preferences, weather, color, and occasion.
- Analyze clothing usage patterns and provide insights.
- Automation of item removal (Arduino) operates with low power consumption.

### **2.2 Constraints**

In parallel with the functional requirements, there are usually constraints that must be taken into account.

#### **2.2.1 Technical constraints**

- Use a camera to detect and classify clothing items entering and exiting the closet.
- Provide outfit suggestions based on user preferences, weather, color, and occasion.
- Offer a user-friendly mobile application for wardrobe management.
- Analyze clothing usage patterns and provide insights.
- Operate with low power consumption.

#### **2.2.2 Non-technical constraints**

- The system should be easy to use.
- The mobile application should be user-friendly.
- Recommendations should match user style and preferences.
- Ensure privacy and data security.
- The solution should be affordable.

### 2.2.3 Standards / codes / regulations / policies

- **Specific:**
  - ISO/IEC 27001 for data security.
  - OpenCV standards for computer vision implementation.
- **General :**
  - Use of UML for system design and documentation.
  - Adherence to BPMN for workflow modelling.

## 3 Background / Research / Related work / Literature review

### 3.1 History of Smart Wardrobe Management Systems:

Wardrobe Management is newly starting to rise with the advancement of some interesting technologies used for this purpose for example: Computer Vision, Artificial Intelligence, Internet of things (IoT).

Prior to these advancements, wardrobe management was dependent on manual organization or basic mobile applications where the user would manually add his/her clothing items to the application.

Some examples of early smart closet management solutions:

- Clothing management apps: Smart Closet – Rabbit tech inc or Cladwell allow users to digitally organize outfits but require manual input of clothing items.
- Camera-Based Smart Mirrors: Companies such as Amazon (Echo Look) and Samsung (Smart Mirror) introduced AI-driven outfit suggestion systems using cameras, but these are mostly standalone devices rather than integrated wardrobe solutions.

With the advancement of needed technology, some predefined models, tools and datasets can be found beneficial for the start of the project.

### 3.2 Current Researchers in the Field

- **Alibaba:**

Alibaba's AI-driven clothing recommendation system: AI-based mix-and-match suggestions, Smart lock to capture consumers' preference [1]

- **Samsung:**

Samsung AirDresser: clothing maintenance solutions on the market

It focuses on refreshing and maintaining clothing but does not track, or recommend outfits [2]

### 3.3 State-of-the-Art Technologies in Smart Closet Systems

- **AI-Powered Outfit Recommendation Systems:**

FashionNet–Classify clothing and match outfits based on user preferences. [3]

- **Computer Vision for Clothing Detection:**

Fashion-MNIST Dataset: Fashion-MNIST is a dataset created by Zalando, consisting of 28x28 grayscale images of fashion products for training AI models to recognize different types of clothing.

DeepFashion2 Dataset: Introduced by Ge et al. in DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images.

- **IoT-Based Wardrobe Systems:**

Wearable RFID Clothing Trackers (Huang et al., 2021) – Uses RFID tags embedded in clothing for tracking, having a high implementation cost.

### 3.4 Global Evaluation of Existing Solutions: Pros, Cons, and Methodology Differences.

| Name Of System                                                          | Functionalities                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Pros                                                                                                                                                                                                                                                                                                                                         | Cons                                                                                                                                                                                                                                                                                                                                   | Key Differences from the Project Smart Closet                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Samsung Family Hub™ Refrigerator</b><br>[4] [5] [6] [7] [8] [9] [10] | <ul style="list-style-type: none"> <li>• Touchscreen display with built-in applications</li> <li>• Food inventory management and tracking</li> <li>• Built-in cameras for remote viewing of contents</li> <li>• Automated grocery ordering integration</li> <li>• Meal planning and recipe suggestions</li> <li>• In-app shopping list and ordering management</li> <li>• Remote control using mobile application and integration with other smart devices</li> <li>• Energy monitoring and efficiency management</li> </ul>                | <ul style="list-style-type: none"> <li>• Large screen puts smart home control at fingertips</li> <li>• Energy monitoring and efficiency management</li> <li>• Integration with the SmartThings app</li> </ul>                                                                                                                                | <ul style="list-style-type: none"> <li>• Application integration needs some work</li> <li>• Screen not completely intuitive</li> <li>• Bixby Voice Assistant Limitations</li> <li>• Expensive Price and Durability Issues</li> </ul>                                                                                                   | <ul style="list-style-type: none"> <li>• Primary Functionality: One focuses on kitchen management, while the other manages wardrobes.</li> <li>• Integration with Other Smart Devices: One serves as a smart home hub, while the other is dedicated only to wardrobe management.</li> <li>• Technological Components: One includes a multifunctional touch screen, while the other features AI-powered outfit suggestions and a mobile app</li> </ul> |
| <b>Lucy: The Smart Closet</b><br>[11] [12] [13]                         | <ul style="list-style-type: none"> <li>• Interactive Smart Mirror with Integrated Applications</li> <li>• Smart Clothing and Organization Tracking</li> <li>• Gesture-Controlled Interface</li> <li>• Clothes Location Tracking</li> <li>• Automated Outfit Planning and Event – Based Recommendations</li> <li>• Outfits History Log</li> <li>• Calendar and Music Integration</li> <li>• Targeted Wardrobe Lighting</li> <li>• Personalized Accounts and Smart Syncing.</li> <li>• Rotating Crutch Bar for Space Optimization.</li> </ul> | <ul style="list-style-type: none"> <li>• Displays a digital wardrobe for easy outfit planning</li> <li>• Tracks clothes automatically</li> <li>• AI-powered outfits based on user preferences and past selections.</li> <li>• Offer seasonal wardrobe updates</li> <li>• Maximizes vertical storage with rotating clothing racks.</li> </ul> | <ul style="list-style-type: none"> <li>• Interface lacks practicality and visual clarity.</li> <li>• No virtual try-on feature</li> <li>• High setup cost for NFC enable wardrobe: each tag ranges from \$1 to \$3. For a wardrobe with 100+ clothing items, the cost can exceed \$100, making it a significant investment.</li> </ul> | <ul style="list-style-type: none"> <li>• Primary Functionality: AI driven Outfit Suggestions: Learns user preference based on weather, events and personal preferences.</li> <li>• One employs NFC based tracking. The other utilizes Computer vision (CV) to identify and differentiate clothing items based on type, color and texture.</li> </ul>                                                                                                  |

|                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ACloset</b><br>[14] [15] [16]<br>[17] | <ul style="list-style-type: none"> <li>• Catalog clothing with AI-assisted tagging and automatic background removal</li> <li>• Track wardrobe insights, including cost-per-wear and usage analytics</li> <li>• AI-generated outfit recommendations based on weather conditions.</li> <li>• Engage with a Social feed for outfit inspiration and sharing.</li> <li>• Buy and sell secondhand clothing through the marketplace.</li> </ul> | <ul style="list-style-type: none"> <li>• Smooth transitions and considerate interface.</li> <li>• AI categorization &amp; background removal</li> <li>• Wardrobe stored in the cloud, allowing access from any device by logging into the same account.</li> <li>• The application's social platform is unique in its attempt to build community and allow users to share outfits and gain inspiration.</li> <li>• Free version available on IOS and Android</li> </ul> | <ul style="list-style-type: none"> <li>• Photo uploading process can be glitchy and may require more than one attempt at times</li> <li>• No inspiration tab for adding photos of outfits liked and building outfits based on them</li> <li>• AI outfit recommendations may not always match perfectly.</li> <li>• The integrated resale marketplace comes with risks: non-secured transactions, most of the items tend to be low value</li> <li>• The social feed, filled with user-generated content may make the applicatin feel cluttered. If a clean and private digital closet is preferred this might not be the best option.</li> </ul> | <ul style="list-style-type: none"> <li>• ACloset requires manual input unlike Smart Closet which uses a camera-based video detection to automatically track the addition and removal of clothing reducing user effort.</li> <li>• ACloset relies on user input with no IoT integration, whereas Smart Closet provides an embedded camera system combined with a user-friendly mobile application offering a fully automated and interactive experience</li> <li>• ACloset depends on cloud storage which increases energy use and requires a subscription starting at \$3.99/month for more than 100 items., but Smart Closet is optimized for sustainability and cost-effectiveness through local processing.</li> <li>• ACloset relies on cloud-based storage, whereas Smart Closet prioritize user data protection by processing and storing information locally ensuring compliance with GDPR standards.</li> </ul> |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Table 1:Comparative Analysis of Existing Smart Systems**

### 3.4.1 Samsung Family Hub™ refrigerators

Samsung Family Hub™ refrigerators keep the family connected anytime, anywhere. The user can share pictures, videos and drawings with Google photos, control smart appliances and devices, get all the benefits of Alexa built-in and quickly add items to shopping list all right from Samsung smart fridge.

The Samsung smart refrigerator features a 21.5-inch touchscreen on the right door, pre-loaded with applications such as Pandora and Ring Security, with the option to add more applications. It operates similarly to a Samsung Galaxy phone, allowing customization and personalized screens for each household member. The 'View Inside' feature provides time-stamped images of the refrigerator's contents, enabling users to scroll through past images for up to three weeks. Users can create a shopping list directly on the screen, add food items manually or by voice using Samsung's voice assistant, Bixby, and track expiration dates. However, Bixby has limitations, requiring users to be very close to the refrigerator and confirm each item by tapping the screen. Additionally, it struggles to recognize certain words. The Shopping List application categorizes frequent items into groups such as bread, fruits, and vegetables and supports various stores, including Cash & Carry, Whole Foods, and Costco. A partnership with Instacart allows users to shop directly from the screen, with the

refrigerator's artificial intelligence vision system detecting when items are running low and adding them to the Instacart application for reordering. The system uses cameras above the refrigerator door to track when items are added or removed but cannot detect items stored in the door bins or freezer. The refrigerator also includes artificial intelligence energy mode, integrated with SmartThings Energy, which analyzes user routines and adjusts connected devices to optimize energy consumption, reducing usage by up to seventy percent.

### **3.4.2 Lucy**

Lucy is an AI-powered walk-in closet designed to lessen the burden of choosing outfits regularly. It features an interactive smart mirror that provides outfit suggestions based on weather, calendar events, and personal preferences based on prior choices or chosen styles, while also displaying real-time information including the time and weather, enhancing user experience. Gesture control allows user to interact with the smart mirror and navigate through the menu seamlessly. Clothes tracking is enabled using NFC tags and weight sensors to ensure an updated inventory of wardrobe items. Users can easily locate a clothing item by name, category, or color, with LED indicators highlighting the exact position. Lucy also maintains an outfit history log where a user can re-visit outfits previously worn to avoid outfit repetition, or for inspiration. Additionally, it syncs with Google Calendar to learn any upcoming event and present event-based outfit planning. It also pairs with Spotify to setup mood-based music to elevate the atmosphere. Additional features to enhance accessibility, include targeted wardrobe lighting and rotating crutch bar that maximizes closet space. Lucy ensures a personalized, efficient, and effortless dressing experience.

### **3.4.3 ACloset**

Acloset, a Digital Closet and Fashion Space Designed to Enhance Users' Fashion Experience, a mobile application designed to assist people who have a large collection of clothing yet struggle to create suitable outfits. With AI-powered technology Acloset offers a structured approach to wardrobe management enabling users to create a personal digital closet and discover effective styling options. Additionally, by selling unwanted clothing and purchasing pre-owned items users can contribute to a more sustainable fashion life that benefits both individuals and the environment.

## **4 Solution design**

### **4.1 Multiple design alternatives comparison**

This section explores three different solutions for implementing a Smart Closet system, establishing the pros and cons of each, and choosing the best solution design for the project while taking into consideration the technical requirements.

The following solutions each have a different technique to track and detect clothing items in the closet.

#### **4.1.1 RFID Tracking Clothes**

This alternative solution uses RFID technology to track items in inventory.

"An RFID (Radio Frequency Identification) uses electromagnetic fields to automatically identify, and track tags attached to objects. An RFID system consists of a tiny radio transponder called a tag, a radio receiver, and a transmitter. When triggered by an electromagnetic interrogation pulse from a nearby RFID reader device, the tag transmits digital data, usually an identifying inventory number, back to the reader. This number can be used to track inventory goods." [18]

RFID tags are attached to pre-registered clothes in the closet, and items are tracked with a RFID reader in the closet.

The user must attach a tag on each of his items, and enter the data related to the item added to the inventory. The RFID cannot recognize automatically the type of clothes or color of the item. Each item is identified by a unique id corresponding to the number of the tag.

A trained AI model will do outfit suggestion based on weather data input, calendar/event input, and previous outfit history.

| Advantages                                               | Disadvantages                                                                                                                               |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Real-time tracking                                       | Very expensive (each item requires an RFID tag and a reader, and the exact price is uncertain.)                                             |
| There is no requirement for lighting or proper exposure. | Excessive Manual input (Tag attachment, clothes entry, manual classification of item features for the AI model - colors, category, texture) |
|                                                          | RFID tags are not washable, so they need to be removed and reattached each time                                                             |

**Table 2:RFID-Based Clothing Tracking: System Overview**

Non-conformity with the technical requirements:

- Complexity of implementation and usage
- No AI based clothing classification / recognition

#### 4.1.2 Manual Input on a Mobile application

This solution does not require any hardware.

A simple mobile application can be implemented at no cost.

The user must manually add pictures of his clothing items to the mobile application.

A trained AI model can extract the features needed from the item's picture uploaded by the user, and suggest outfit based on the weather/ event/ user preferences.

When choosing the outfit of the day, the user has to remove the item from the virtual closet on his mobile application, to keep his/her inventory up to date.

| Advantages                          | Disadvantages                                                                                  |
|-------------------------------------|------------------------------------------------------------------------------------------------|
| Simple Implementation (No Hardware) | Excessive Manual input                                                                         |
| Low / No cost                       | No real-time tracking of items in the inventory                                                |
|                                     | Need to correctly capture a picture of the piece of clothes according to a certain methodology |

**Table 3:Manual Clothing Inventory through Mobile app: System Overview**

Non-conformity with the technical requirements:

- Not user friendly
- No automation

#### 4.1.3 Camera Based Clothes tracking with AI recognition

This solution alternative requires little to no hardware, a trained AI model and a mobile application.

A small tablet with a camera integrated is needed for real-time tracking.

The opening of the closet triggers the camera's launch. This automation makes it easier on the user than manual removal method. The camera and the trained AI model can detect and classify clothes and identify added or removed clothes from the clothes.

The AI model give outfit suggestion for the user based on his/her preferences, weather and occasion.

This implementation is accompanied by a mobile application with a simple user interface, where the user can navigate through the item in his/her inventory and through the possible outfit he/she can wear today.

To enhance flexibility, the user can manually upload pictures if desired. In case the closet is implemented in a low-light environment, a flashlight feature is included to improve the accuracy of the system.

| Advantages                                      | Disadvantages                              |
|-------------------------------------------------|--------------------------------------------|
| Real-time inventory tracking + Automation (IOT) |                                            |
| Automated System / Little manual work by user   | Complex Implementation (AI model training) |
| Scalable                                        |                                            |
| User friendly                                   |                                            |

**Table 4:AI-Powered Camera-Based Clothing Tracking:System Overview**

Conformity with the technical requirements:

- Very User friendly
- AI based inventory tracking and outfit recommendation
- Use a camera to detect and classify clothing items
- Automated and operates with low power consumption

**Final Decision:** The best solution design for the smart closet is Camera Based Clothes tracking with AI recognition as it meets all the technical requirements and does not present major disadvantages.

## 4.2 Problem formulation

Managing a personal wardrobe can be challenging, especially when trying to organize clothing, plan outfits, or track clothing usage. A smart closet system can simplify this process by leveraging camera-based video detection, AI algorithms, and a local mobile application to help users organize, track, and optimize their wardrobes. This project aims to create an intelligent, user-friendly solution for personal wardrobe management using computer vision and AI. The primary goal of this project is to design and implement a smart closet system that uses camera-based video detection and AI to help users manage their wardrobes efficiently.



Specific objectives include:

1. Camera-Based Clothing Tracking: Use a camera to detect when clothing items are added to or removed from the closet.
2. Segmentation
3. Outfit Suggestions: Develop an AI algorithm to suggest outfits based on user preferences, weather, and occasion.
4. Local Mobile Application: Create a mobile application for users to interact with the system, view their wardrobe, and receive outfit suggestions.
5. Data Analytics: Analyze clothing usage patterns to provide insights (for example, most/least worn items).
6. Energy Efficiency: Ensure the system operates efficiently with low power consumption, while still being automated with an Arduino board.

## **4.3 Required resources**

### **4.3.1 Software Resources :**

- Android Studio: for developing the mobile application
- Roboflow: for datasets
- Pytorch, YOLOv8: for image processing and object detection
- PostgreSQL : for database management
- Google Colab : for code implementation and training
- Arduino IDE

### **4.3.2 Hardware Resources :**

- Tablet (Primary Device)
  - Runs the Smart Closet mobile app
  - Uses the built-in camera for scanning and recognizing clothing items
  - Provides a touchscreen interface for selecting and managing outfits
- Built-in Camera (on the Tablet)
  - detects when clothing items are added to or removed from the closet.
- Closet to incorporate the tablet and test the application
- Arduino Board and other necessary components for automation

## **4.4 Conceptual model**

The smart Closet System is designed to help users manage their wardrobes using camera-based detection, AI algorithms, and a mobile application. The system automates clothing tracking, provides outfit suggestions based on user preference, events and weather. It additionally offers insights into wardrobe usage patterns, while ensuring low power consumption and user-friendly interaction.

Below is a structured conceptual model to provide a clear visualization of the system's functionality:

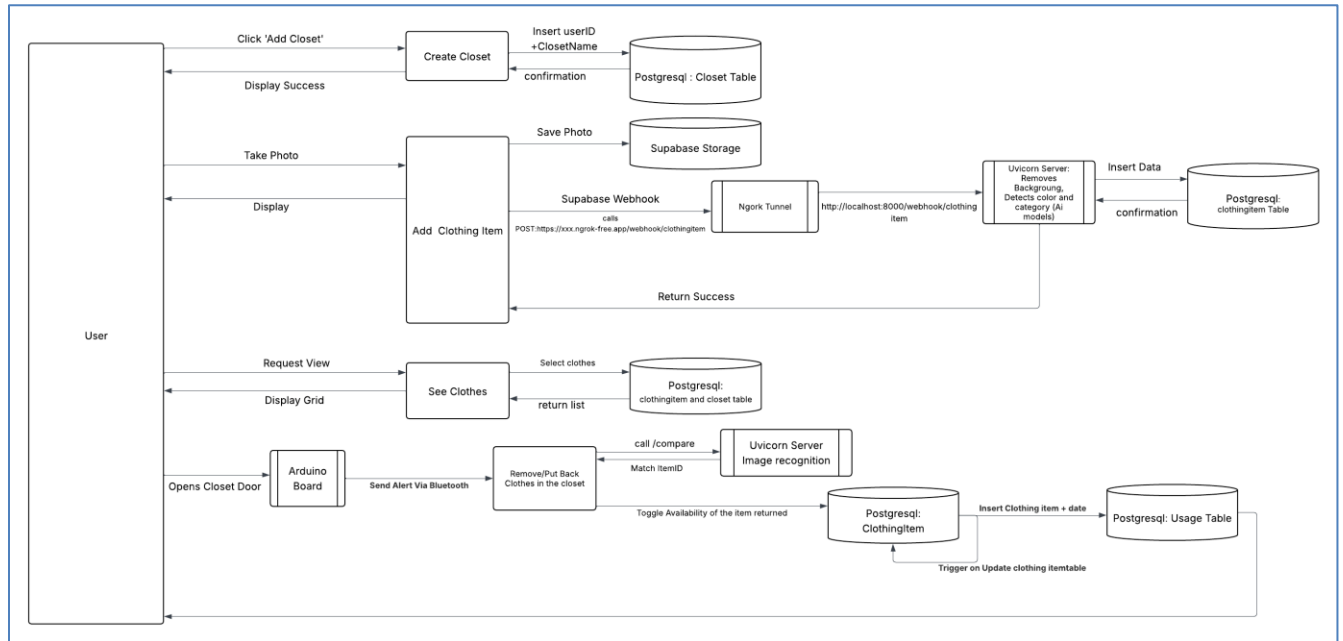


Figure 1:Closet Management System: Conceptual Model 1

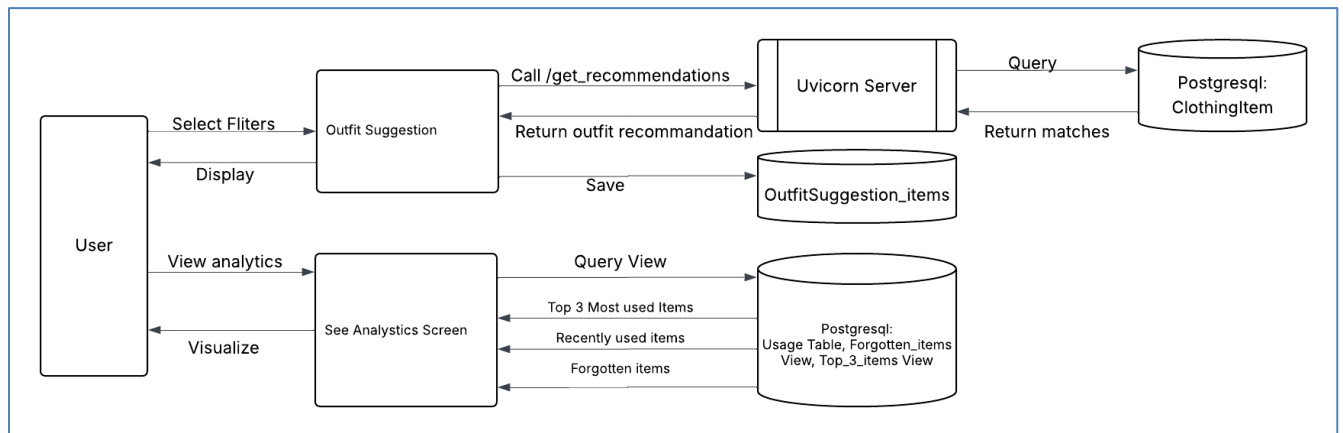


Figure 2:Closet Management System: Conceptual Model 2

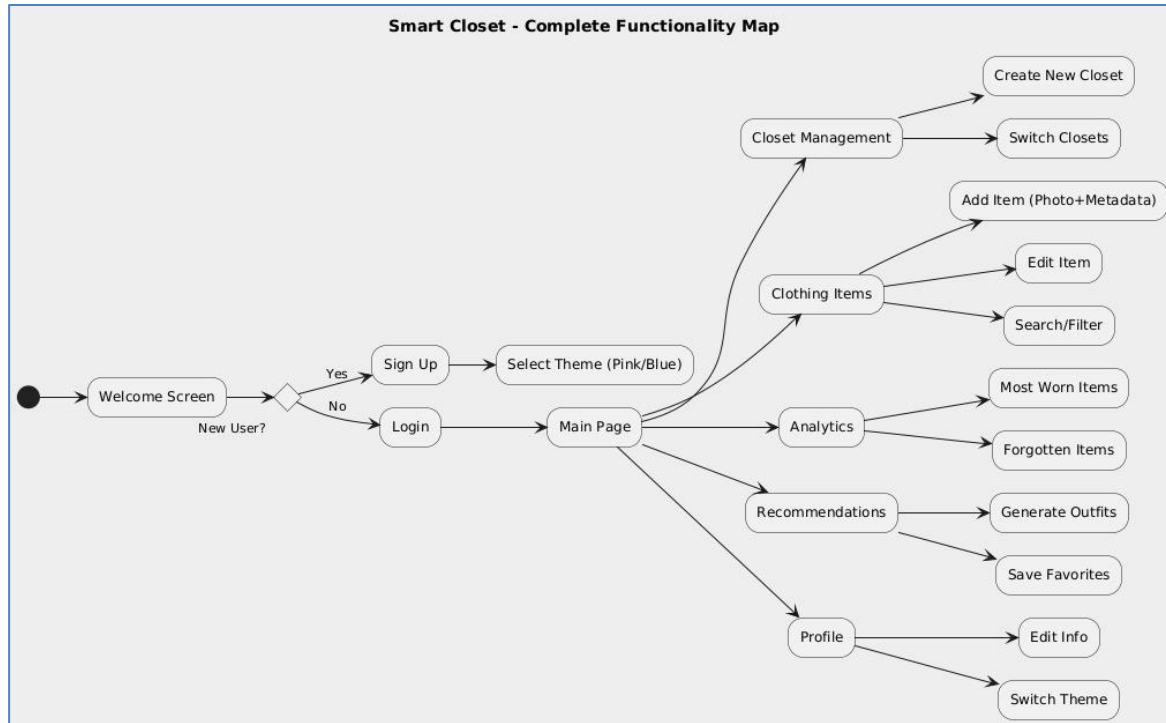


Figure 3:Complete Functionality Map - UML diagram

## 5 Project Management

### 5.1 Planning

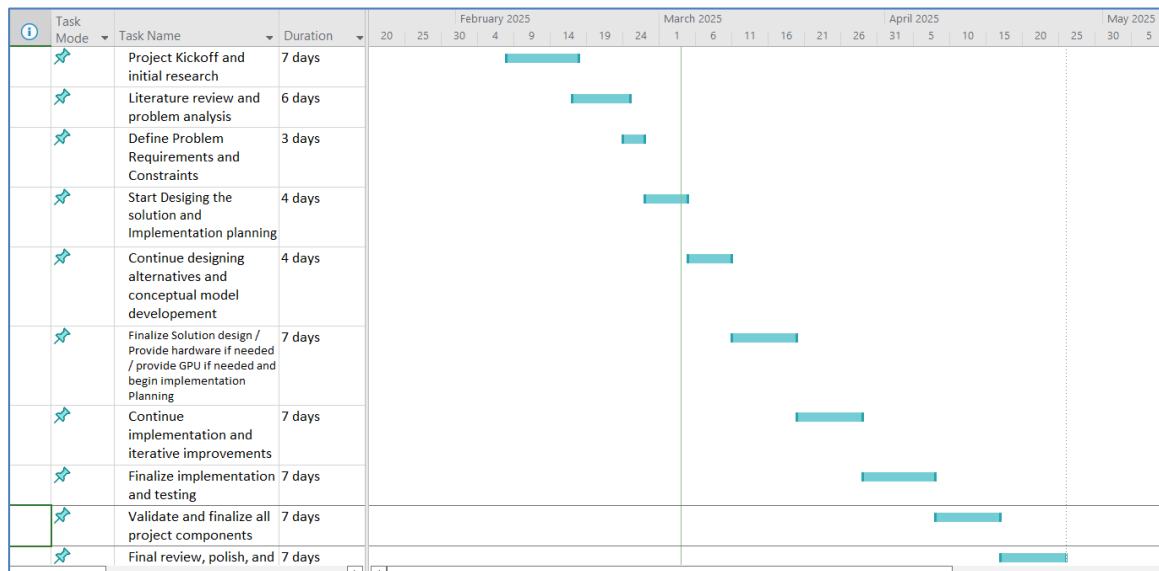


Figure 4:Gantt chart (on Microsoft Project)

## 5.2 Project Facts and Figures

List all the project members, their disciplines, and the topic they will be working on.

| Name               | Affiliation | Topic                                     |
|--------------------|-------------|-------------------------------------------|
| Karen Khoury       | CCE         | Mobile Development / Backend Server / IOT |
| Lea Chamseddine    | CCE         | AI Implementation                         |
| Fatima al Rabih    | CCE         | AI Implementation                         |
| Marilyn Abou Tayeh | CCE         | Mobile Development                        |

Table 5: Project Team Composition Member Roles and Responsibilities

## 5.3 Conflict Management [19]

Effective conflict resolution will be essential for maintaining team cohesion and project efficiency. The following strategies will be employed:

- **Encouraging Open Dialogue and Active Listening:** Every team member will have the opportunity to share thoughts and concerns. Active listening will help clarify misunderstandings and strengthen team dynamics.
- **Implementing Collaborative Problem-Solving Techniques:** When disagreements arise, especially regarding technical choices, discussions will be held to compare options and determine the best solution.
- **Leveraging Conflict Management Styles:** Minor issues will be addressed through compromise, while critical decisions will require a direct and structured approach.
- **Focusing on Emotional Intelligence:** Recognizing frustrations early and addressing them proactively will help maintain a positive and productive environment.
- **Engaging a Neutral Third Party When Needed:** If conflicts become deeply established, the supervisor will act as a mediator to guide discussions toward resolution. For technical challenges beyond the team's expertise, additional support will be sought from expert instructors.

## 6 Solution Implementation

### 6.1 Database Management

Database management for Smart Closet relies on a relational Database hosted on Supabase, which is an open-source backend solution based on Postgres Databases.

#### 6.1.1 Table Conception:

This relational database consists of eight tables:

1. **User** (user\_id PK, name, gender, job, home\_location, birth\_date)
2. **Closet** (closet\_id PK, user\_id FK, closet\_name, size, capacity)
3. **ClothingItem** (item\_id PK, closet\_id FK, name, category, color, material, season, last\_worn, image\_url, style)
4. **OutfitRecommendation** (recommendation\_id PK, user\_id FK, weather\_condition\_id FK, style)

5. **OutfitRecommendation\_Items** (recommendation\_id FK, item\_id FK, category, PK(recommendation\_id, item\_id))
6. **WeatherData** (weather\_id PK, temperature, condition, date\_time, location)
7. **Usage** (usage\_id PK, user\_id FK, item\_id FK, worn\_date, event\_id FK, weather\_condition\_id FK)
8. **EventCalendar** (event\_id PK, user\_id FK, event\_type, event\_date, location)

**Views:**

1. **ForgottenItemsView** (item\_id PK, closet\_id FK, name, category, color, material, season, last\_worn, image\_url, style, availability, user\_id FK)
2. **TopUsedItemsView** (item\_id PK, name, image\_url, count, rank, user\_id FK)

This database represents the logic behind the clothes management system. Each user can have many closets (ex: parents can manage the closet of their children, or a person can have multiple houses implying multiple closets).

Recommended outfits could be generated using the WeatherData Table when integrated to the algorithm. A recommendation yields multiple items so the usage of Outfit recommendation\_Item table is necessary.

The table Usage track when users are wearing those clothes and generate Statistics of each user and his clothes preferences,

Those statistics are stored in 2 views to facilitate Supabase/Kotlin queries for data analysis.

**6.1.2 Security and Authentication:**

The Authentication process is assured by Supabase authentication, providing a secure system for session management. Each user can only access his own data thanks to Row-Level Security (RLS) activated on chosen tables.

Personalized policies can be defined:

- Allow only authenticated users to insert, read, or modify their own records.
- Restrict write access to certain sensitive fields

**6.1.2.1 Communication with the mobile app:**

The mobile application interacts with the database through APIs REST generated automatically by Supabase. Those endpoints can be called by the mobile application used the SDK supabase-kt .

The application also retrieves data from custom views using the same syntax, which makes querying complex logic straightforward while keeping the mobile code clean and efficient. All data is parsed into data models (DataClasses) using Kotlin serialization, and displayed reactively in the UI through StateFlow and ViewModel. (please refer to Appendix 10.1)

**6.1.2.2 Triggers and Webhooks:**

A Supabase WebHook is tied to the table “clothingitem” to automatically call a FastAPI from the backend, every time an item is inserted. This system allows us to process the image being inserted to the database by removing the background, extract colors, and detecting the clothes category using the machine learning algorithms.

Another SQL trigger is set up on the “clothingitem” table. When the availability of an item goes from one to zero, the trigger inserts this item into the table Usage along with the corresponding date. This table is used then to generate views about most/least used items.

### 6.1.2.3 Techniques, Tools, and Skills Applied

The project uses SQL language for working with relational database. Supabase Studio was used, which is a web interface to manage tables, data, and views. For security, Supabase provides Row-Level Security (RLS) / custom policies for controlling access to data.

Communication between database and other parts such as FastAPI or mobile application is done using REST API, and Supabase webhooks/ SQL triggers for automation.

## 6.2 Mobile Application Backend

Mobile backend is based on two complementary components:

- Supabase Client, integrated to the mobile application for data management
- FastAPI server, constituting the intelligent part of the project.

### 6.2.1 Supabase:

#### 6.2.1.1 Supabase Client

The Android application uses the supabase-kt SDK, which allows direct access to Supabase services from Kotlin. This embedded client acts as a backend within the mobile application.

The main operations handled through Supabase are:

- Authentication (sign-up, sign-in) using the authentication module,
- Reading/writing to the PostgreSQL database using from("table") queries,
- File management (uploading, retrieving URLs) through Supabase Storage.

The Supabase client is initialized at the launch of the application using a provided URL and public key from the Supabase project. (please refer to Appendix 10.2)

Session persistence is automatically handled on the client side. On startup, the application checks if an active session is available, in order to direct the user to the login screen or directly to the user interface.

#### 6.2.1.2 Supabase Storage

Supabase Storage is a service that securely store and manage files such as images, videos, or documents in buckets. It provides built-in access control and integrates seamlessly with Supabase authentication and database permissions.

In this project, two buckets are used:

- picture-clothes: This bucket contains the picture of the item before and after background removal. Photos have a public URL, stored in the clothingitems tables allowing the display the image of the item in the mobile application.
- twosecPic : This bucket stores the images of items being removed/put back into the closet. Those photos are needed in order to track clothing availability. A script, comparing the specific image to the images in picture-clothes bucket, updates the availability of the corresponding item.

#### 6.2.1.3 Mobile architecture:

The mobile architecture follows a Model-View-ViewModel (MVVM) logic:

- ViewModels that use StateFlow to expose data to the view,
- Repositories that centralize access to the Supabase client
- A NavHost that manages navigation between screens.

#### 6.2.1.4 Data Flow

| User Action                           | Application Action                                                         | Supabase Involved                                                                                      | FastAPI Involved                                                                          |
|---------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Add closet                            | Creates new closet with userid                                             | Insert new closet in closet table                                                                      | -                                                                                         |
| Add Clothing item                     | Taking Picture, Inserting item                                             | Insert photo in Supabase storage and item in table clothingitem                                        | Webhook triggering /webhook/clothing item<br>Removing bg and detecting category and color |
| See Clothes                           | Display all available items for the connected user                         | Select clothes from clothingitem and closet table                                                      | -                                                                                         |
| Remove/Put Back Clothes in the closet | Open camera for user to take a picture of the item in hand                 | Toggle Availability of the identified item                                                             | Endpoint /compare to identify the removed/add item against the user's inventory           |
| Outfit Suggestion                     | Display Suggested items as the user choses the features (season, style...) | Insert suggested items in table outfitSuggestion_Items                                                 | Endpoint /get_recommendations                                                             |
| See Analytics Screen                  | Display Usage Calendar , podium and Used/forgotten items                   | Query View and Usage Tables to get :Top 3<br>Most used Items<br>Recently used items<br>Forgotten items | -                                                                                         |

**Table 6:Applications Workflow: User Interactions and System Responses**

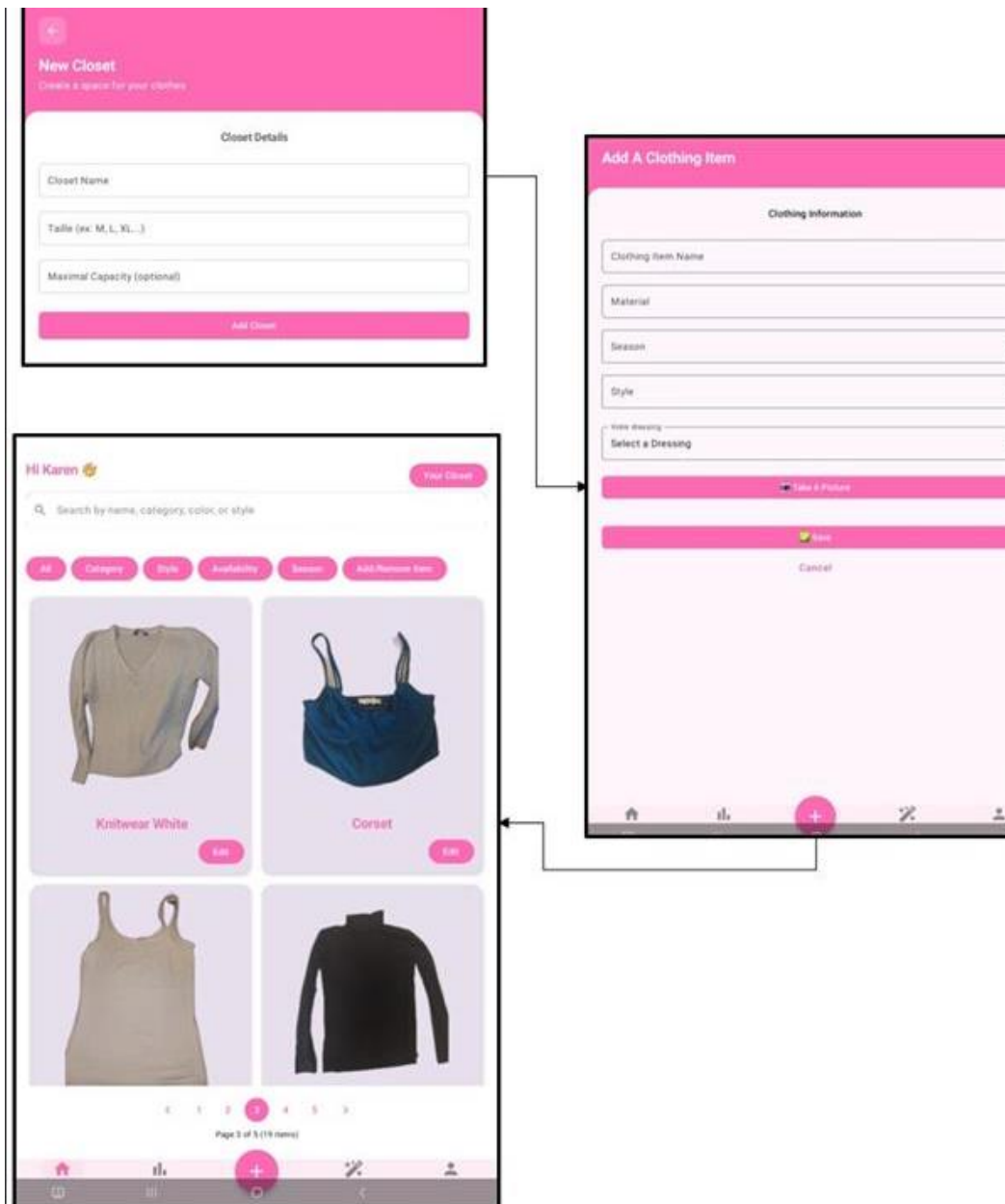


Figure 5: User flow for adding a new closet and clothing items, with photo upload and categorized display in the digital wardrobe.



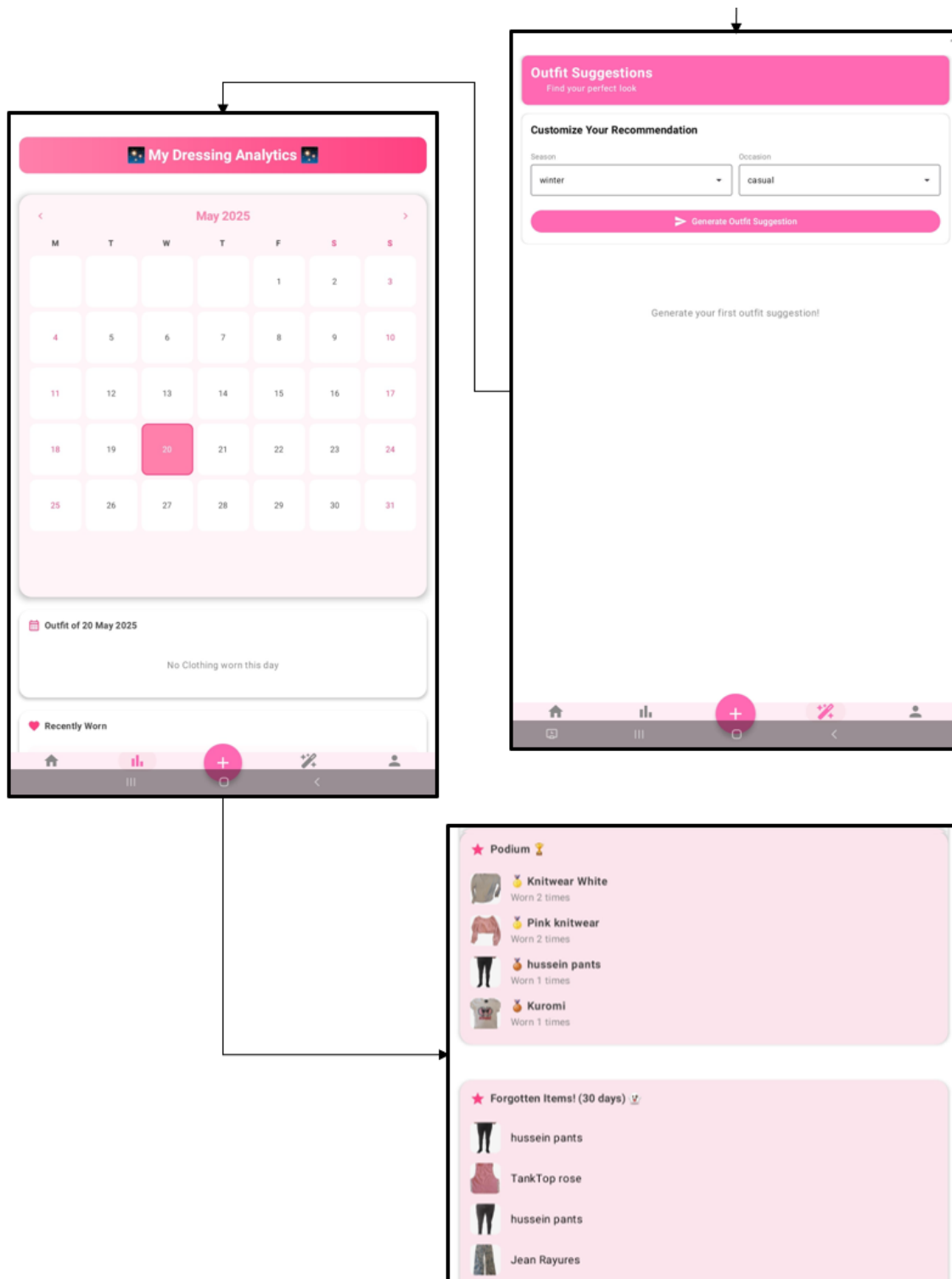


Figure 6: Analytics and recommendation flow showing outfit history, frequently worn items, and personalized outfit suggestions.

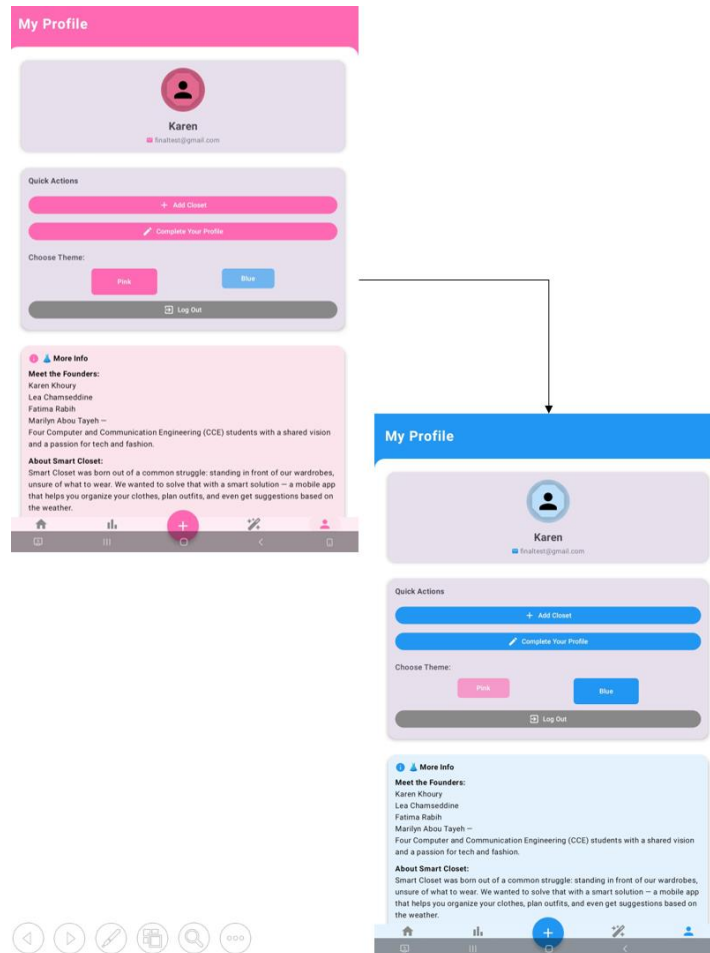


Figure 7: Profile UI illustrating theme switch from pink to blue based on user preference.

All of the screens can be displayed in Pink or Blue, following to the user's choice.

## 6.2.2 FastAPI External Server

### 6.2.2.1 Architecture and technologies

The backend was developed using python with the framework FastAPI, known for its fast execution time and its compatibility with OpenAPI's standards and its easy deployment.

The backend exposes many REST endpoints called by the mobile application directly or through Supabase's webhooks.

Backend responsibilities are:

- Image segmentation (remove background)
- Uploading processed images to Supabase Storage
- Analyzing clothes added to the closet (category and color)
- Getting Outfit recommendation

Thus, when a user adds a clothing item, an image is taken using the application's camera and then uploaded to Supabase Storage. A webhook detects this new insertion and calls a FastAPI endpoint (please refer to Appendix 10.3)

The backend downloads the image, applies a background segmentation model using the rembg library, then re-uploads the cleaned image to Supabase Storage, updating the corresponding row in the clothingitem table.

#### 6.2.2.2 Resources Deployment and Communication:

Throughout the development phase, the backend was made public using NGROK.

This tunneling tool creates a bridge between the local machine (localhost) and the public internet allowing communication between the backend and Supabase webhooks and mobile app. (please refer to Appendix 10.4)

The URL generated by ngrok is then inserted in the webhook configuration on Supabase, permitting reception of POST requests on the server when the webhook on Supabase is triggered.

#### 6.2.2.3 Technical Environment:

The languages used are Python 3.11 and kotlin, with FastAPI and Jetpack Compose frameworks. For the local server, Uvicorn was used, and for the communication, supabase-py and ngrok were adopted.

### 6.3 User Interface Implementation

The mobile application's user interface was developed using Jetpack Compose, Android's modern UI toolkit. This method allowed for a smooth and modular development process where the UI reacts automatically to data changes, without need for XML layouts or manual refresh logic.

The application follows the MVVM architecture, enabling clear separation between logic and display. The UI responds to state changes in real-time using Kotlin's Stateflow, while navigation between screens is handled by NavHost.

#### 6.3.1 User Interface Structure

The application is divided into several screens, each with its own purpose:

- **Welcome & Authentication Screens:** The user is asked to create an account, with necessary information required, or if an account is already created, the user can login into their account using email and password.
- **Main Page:** Once authenticated, the users are redirected to the main page, which contains a navigation bar at the bottom, where different screens are available, and they choose what action to take next.
- **Closet List Screen:** This screen displays all items from the selected closet, with options to filter by category, style, and availability. Additionally, a search bar is available to find specific items easily. Users can also have multiple closets linked to a single account, therefore they can switch closets through this screen. The display is paginated using LazyColumn to ensure smooth performance even with large wardrobes.
- **Closet Analytics Screen:** Offers visual insights such as most worn items, recently used clothes, and forgotten items for pieces not worn for more than 30 days.
- **Add Item Screen:** Allows capturing or uploading an image, entering details such as name and category, then submitting to the desired closet. This triggers a webhook to the backend for automation processing.
- **Recommendation Screen:** Displays outfit suggestion based on selected filters such as event type and style.
- **Profile Screen:** Users can complete any missing fields or edit existing information. The screen also includes a theme selector(Pink/Blue), and the selected theme is saved to Supabase and applied across the app.

- **Add Closet Screen:** Users can create multiple closets, for example for different homes or family members. Once a closet is created, the user can switch between closets through a dropdown menu existing in the Closet List Screen.
- **Edit Clothing Item Screen:** Users can edit clothing details such as name, material, color, and style. The screen is pre-filled for a better user-experience.
- **Theme Customization:** The application offers two visual themes: Pink and Blue. On sign up, the users choose one of the two themes, and the selected theme will be stored in the database and applied app-wide. The user can toggle between themes anytime through the Profile Screen.

### 6.3.2 Tools and Techniques used

- **Jetpack Compose:** Used for building composable and reactive UI components.
- **Material3 Components:** Ensure consistent design.
- **Navigation Compose:** Simplifies navigation between composable screens.
- **LazyColumn + Paging:** Enables efficient scrolling for large lists of clothing items.
- **OutlinedTextField + Icons:** For search bars and form fields.

The final interface aims to be clean, user-friendly, and visually appealing. The users can get the full seamless experience that hides complexity of backend processes.

## 6.4 IOT Automation

### 6.4.1 Mechanism

In order to make SmartCloset easier, automation for inventory tracking is needed.

An Ultrasonic Sensor is placed at the back of the closet, measuring continuously the distance between the back and the door of the closet. When the door is opened for the user to take/put back an item, this distance exceeds 50 cm. An Alert is then sent through Bluetooth from Arduino to the tablet.

The mobile application, which is authorized to detect nearby devices, receives this Bluetooth alert and automatically triggers the tablet's camera. This allows the user to quickly take a photo of the item in hand, without wasting time, ensuring that the inventory is always up to date.

Additionally, to improve visibility inside the closet, RGB LED strips are placed inside the closet. These lights only turn on when the closet is detected as open. To control a 12V RGB LED strip safely with Arduino UNO, a special circuit is needed: Three IRF540N N-channel MOSFETs (Q1, Q2, Q3) are used to handle the high current required by each color channel (Red, Green, Blue). Each MOSFET gate is connected to a PWM-capable pin on the Arduino, allowing us to modulate brightness using `analogWrite()` in code.

The RGB LED strip is powered through a separate 12V power supply, and the common anode (RGBLED-CA) is connected to VCC. Protection diodes (D1–D4) was added and a 1000µF capacitor (C3) to stabilize the voltage and prevent drops during color transitions. This setup isolates the Arduino from the high-power circuit while ensuring control of lighting effects. (Please refer to the circuit in Appendix 10.5)

### 6.4.2 Hardware

The hardware needed for the smart closet are:

- Arduino Uno
- SoftwareSerial.h library to create a second serial port and communicate with the HC-05 Bluetooth module.
- Ultrasonic sensor (HC-SR04) for the detection of door movement
- RGB LED strips (12V) to control the three IRF540N
- External 12V source,
- Diodes
- 1000 $\mu$ F capacitor.

## 6.5 Artificial Intelligence Implementation

### 6.5.1 System Architecture

#### 6.5.1.1 Solution components

The AI subsystem comprises three core modules:

- **Image Processing Module:**  
Performs clothing isolation and segmentation, ensures standardized input for following modules, detects clothing items using a YOLOv8-based object detector fine-tuned on a fashion dataset (producing bounding boxes and class labels such as shirt, pants, dress and many more), and extracts dominant colors using segmentation and clustering techniques. These colors are matched to predefined names (for example, blue, red, beige) using a CSV-based hexadecimal database to enhance visual similarity analysis.
- **Visual Similarity Detection Engine:**  
Implements computer vision-based item matching and enables "find similar items" functionality.
- **Intelligent Recommendation System:**  
A hybrid approach combining metadata analysis (season, occasion, category), color theory principles, and probabilistic selection.

#### 6.5.1.2 Technical Infrastructure

In the project, the system was divided into several main components, each with a specific purpose and set of tools. For computer vision tasks involving image processing and feature extraction, libraries such as OpenCV and scikit-image were employed. Data processing, particularly for handling and analyzing metadata, was conducted using pandas. Background removal was addressed with a focus on precise garment segmentation, utilizing the rembg library to achieve clean and accurate results.

The object detection model, centered on clothing detection and classification, was implemented using the Ultralytics YOLOv8 model in conjunction with torch for deep learning tasks. For color classification aimed at extracting the dominant color of garments OpenCV, numpy, and scikit-learn's KMeans clustering algorithm were applied, along with matplotlib for visualization and pandas for organizing the results into tables.

| Component              | Purpose                                 | Key Libraries/Tools                                       |
|------------------------|-----------------------------------------|-----------------------------------------------------------|
| Computer Vision        | Image processing and feature extraction | OpenCV, scikit-image                                      |
| Data Processing        | Metadata handling and analysis          | pandas                                                    |
| Background Removal     | Precise garment segmentation            | rembg                                                     |
| Object Detection Model | Clothing detection and classification   | ultralytics (YOLOv8), torch                               |
| Color Classification   | Domiant color extraction                | OpenCV, numpy , scikit-learn (KMeans), matplotlib, pandas |

**Table 7: Key Components and Tools for Clothing Detection and Segmentation**

### 6.5.2 Image Pre-Processing Techniques

The captured images to be inputted into the application need to pass through many steps in order to meet the application objectives, which are as follows: Image Segmentation, Background removal, adaptive cropping, dimension standardization, and canvas normalization.

#### 6.5.2.1 Image segmentation

The system implements a robust image preprocessing pipeline to isolate garments for subsequent analysis. This multi-stage normalization process ensures consistent feature extraction and comparison.

#### 6.5.2.2 Background Removal

The system employs the rembg library, which leverages a U<sup>2</sup>-Net neural network for high-precision foreground extraction. This approach effectively handles complex textures, folds, and varying lighting conditions, ensuring accurate segmentation of clothing items from their backgrounds.

#### 6.5.2.3 Adaptive Cropping

An intelligent bounding box detection algorithm dynamically crops images to remove extraneous background pixels. The autocrop\_image() function preserves the garment's aspect ratio while offering configurable padding to maintain edge details.

#### 6.5.2.4 Dimension Standardization

Images are resized using Lanczos interpolation, a high-quality method that minimizes distortion during scaling. The resize\_image() function ensures consistent dimensions while preserving critical visual features for downstream analysis.

#### 6.5.2.5 Canvas Normalization

Processed images are centered on standardized backgrounds (transparent) through the resize\_canvas() function. This step guarantees uniformity across the dataset, eliminating variability that could skew comparison results.

### 6.5.3 Visual Similarity Detection Techniques

The system utilizes a feature-based approach to compare clothing items, ensuring robustness against variations in lighting, pose, and minor deformations. Features should be extracted, matched and scored before being fed to the recommendation engine and finally filtered.

#### **6.5.3.1 Feature Extraction**

The Oriented FAST and Rotated BRIEF (ORB) algorithm detects key points and computes binary descriptors, offering an optimal balance between computational efficiency and discriminative power. This method is particularly effective for textile patterns and clothing structures.

#### **6.5.3.2 Feature Matching**

A brute-force matcher with Hamming distance quantifies descriptor similarity. Matches exceeding a threshold distance of 50 are filtered out to reduce false positives and improve reliability.

#### **6.5.3.3 Similarity Scoring**

The system calculates a normalized similarity score as the ratio of strong matches to total key points. This metric provides an interpretable percentage, enabling quantitative comparisons between garments.

This methodology outperforms pixel-wise techniques by focusing on structural patterns rather than exact color or texture alignment, making it resilient to real-world imaging conditions.

#### **6.5.3.4 Recommendation Engine**

The recommendation engine synthesizes rule-based metadata filtering and color harmony principles to generate contextually appropriate outfit suggestions.

#### **6.5.3.5 Metadata Filtering**

User-provided constraints (season, occasion) are applied to narrow the selection pool. The system intelligently handles transitional seasons (for example, summer/winter) by relaxing strict weather-based filters when appropriate.

#### **6.5.3.6 Color Coordination**

A predefined color compatibility matrix maps dominant colors and suggests harmonizing combinations based on fashion theory. For example, red tops are paired with black or white bottoms, while beige coordinates with blue or brown. This logic is implemented through the `get_color_combinations()` function, which includes fallback options for unmatched colors.

### **6.5.4 Object Detection Model**

The core detection model is based on the YOLOv8 (You Only Look Once version 8) architecture, optimized for real-time image recognition and clothing classification. [22]

#### **6.5.4.1 Technical Environment**

The setup for running the YOLOv8 model requires specific hardware, software, and external services. For hardware, an NVIDIA GPU (for example, Tesla T4 or V100) with CUDA support is necessary to accelerate model training and inference. The system should have 12-16 GB of RAM and at least 50 GB of disk space for optimal performance. In terms of software, Google Colab is used as the platform, with Python 3.10+ as the programming language. The essential libraries and frameworks include ultralytics for defining and training the YOLOv8 model, torch (PyTorch 2.x) with CUDA acceleration for deep learning computations, the Roboflow Python library for downloading datasets, and numpy and opencv-python for data manipulation and visualization. Additionally,

the Roboflow API, with a securely managed API key, is required to retrieve the hosted dataset for training and testing the model.

#### 6.5.4.2 Dataset Preparation

The dataset *Main-Fashion* (version 5) was sourced from Roboflow and downloaded using a specific command. For code snippets, please refer to Appendix 10.6.

The resulting folder structure contains:

- train/images/ and train/labels/
- valid/images/ and valid/labels/
- test/images/

Importing the dataset from Roboflow comes with a data.yaml file, which contains essential metadata about the dataset, such as the paths to the training and validation data, the number of classes, and the class names. This configuration file ensures that the dataset is properly structured and can be easily used for model training. In addition, data.yaml file contains class labels such as 'pants', 'shirt', and others. These class labels are crucial for the prediction process, as they define the categories the model will learn to recognize. When the model is trained, it uses these class labels to classify new, unseen data into the correct category.

An illustration of a data.yaml file:

```
!cat /content/Main-Fashion-2/data.yaml

names:
- Tshirt
- dress
- jacket
- pants
- shirt
- short
- skirt
- sweater
nc: 8
roboflow:
  license: MIT
  project: main-fashion-tkl6c
  url: https://universe.roboflow.com/smartcloset/main-fashion-tkl6c/dataset/2
  version: 2
  workspace: smartcloset
test: ../test/images
train: ../train/images
val: ../valid/images
```

Figure 8:YAML configuration file for a fashion object detection dataset, defining class labels and dataset structure.

#### 6.5.4.3 Model Selection & Training

- **Base Model:** YOLOv8x.pt is the extra-large variant of the YOLOv8 model, designed for high-capacity feature extraction. The "x" in YOLOv8x indicates that this model is optimized for more complex tasks and larger datasets compared to smaller versions of YOLO. The extra-large variant of the model has more parameters and a deeper architecture, allowing it to capture more complex features from input data, making it suitable for large, challenging datasets or tasks requiring high accuracy.



- **Hyperparameters:**

```
!yolo task=detect mode=train model=yolov8x.pt data="/content/Main-Fashion-5/data.yaml" epochs=50 imgsz=640 batch=16 amp=True lr0=0.01 lrf=0.1
momentum=0.937 weight_decay=0.0005
```

**Figure 9: YOLO model training configuration with dataset and hyperparameters**

- **Epochs (30):** Controls how many times the model will see the full training data.
- **Image Size (640 × 640):** The resolution of the images affects the model's ability to detect small or intricate features. Higher resolution usually leads to better accuracy but requires more computational resources.
- **Batch Size (16):** A balanced batch size that ensures efficient memory utilization and stable updates.
- **Learning Rate (0.01, factor 0.1):** Controls how quickly the model learns, with a gradual reduction towards the end to fine-tune the model.
- **Momentum (0.937):** Speeds up convergence by incorporating previous updates into the current gradient update.
- **Weight Decay (0.0005):** Helps prevent overfitting by regularizing the weights.
- **Mixed Precision (amp=True):** Accelerates training and reduces memory usage by using 16-bit precision where possible, optimizing GPU usage.

The YOLO framework automatically ignored the manually set `lr0` (learning rate) and momentum values because the `optimize=auto` mode was enabled. This mode is designed to automatically determine the optimal optimizer, learning rate, and momentum values to enhance the training process.

Additionally, the framework selected the **AdamW optimizer** with specific weight decay parameters. This choice is commonly made for certain datasets and tasks, as AdamW is known to improve training stability and convergence speed.

This fine-tuning adapts the pretrained weights to the domain-specific fashion classes, balancing convergence speed and overfitting prevention.

#### 6.5.4.4 Validation and Evaluation:

The evaluation output provides a comprehensive analysis of the YOLOv8 model's performance on a validation dataset, emphasizing both the technical details and the results of the model's evaluation. This YOLOv8 model consists of 112 layers, with 68,131,272 trainable parameters, and operates with a computational complexity of 257.4 GFLOPs.

The validation dataset comprises 447 images, each labeled with no backgrounds or corrupt data. Metrics such as Precision ( $P = 0.752$ ), Recall ( $R = 0.775$ ), and Mean Average Precision ( $mAP50 = 0.841$ ) offer insights into the model's performance, with an  $mAP50-95$  of 0.841 indicating consistent performance across varying Intersection over Union (IoU) thresholds.

Class-specific analysis reveals high-performing categories such as "dress" ( $mAP50 = 0.962$ ) and "pants" ( $mAP50 = 0.946$ ), whereas "shirt" shows relatively lower performance with an  $mAP50$  of 0.610, indicating areas for potential improvement.

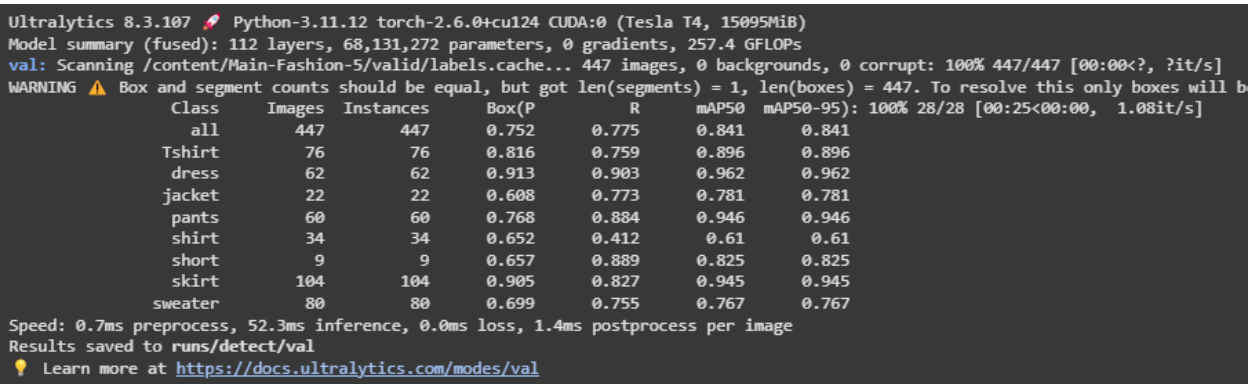


Figure 10:Accuracy Metrics for YOLOv8 (Section 6.5.4)

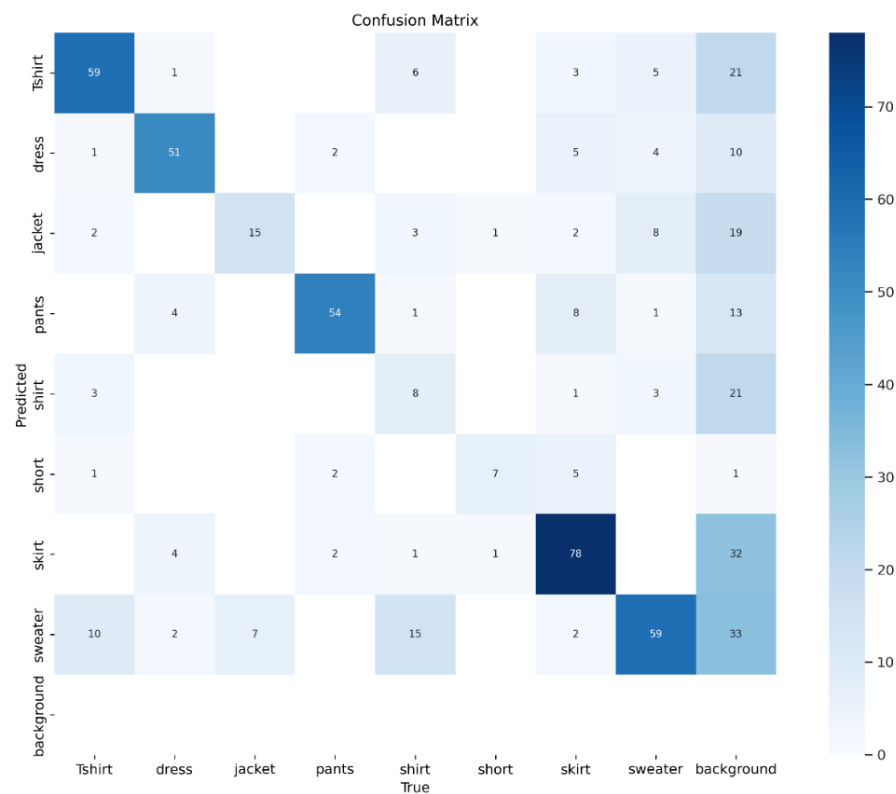


Figure 11:Confusion Matrix

The confusion matrix (Figure 9), a crucial tool in this evaluation, provides a detailed visualization of true positive, true negative, false positive, and false negative predictions across all classes. This matrix helps identify which classes are often misclassified and offers a clear path for optimizing the model by highlighting inconsistencies in predictions. The confusion matrix quantifies classification performance across eight apparel categories. Diagonal elements indicate correct predictions, while off-diagonal values reveal systematic misclassifications. The matrix shows strong diagonal dominance, confirming baseline recognition capability. However, significant off-diagonal concentrations highlight persistent challenges in distinguishing visually similar categories and suppressing background interference. These patterns suggest the need for targeted dataset augmentation to address ambiguous cases and enhanced preprocessing to mitigate background noise. The results provide clear priorities for model refinement while demonstrating fundamental competency in garment classification.

#### 6.5.4.5 Inference Pipeline:

After validation, the trained model is used for making predictions on new test data. Predicted images with bounding boxes are saved for further review and qualitative analysis, which involves visually inspecting the predictions to assess the accuracy, detect misclassifications, and identify strengths and weaknesses of the model's performance.

#### 6.5.4.6 Required Resources

To run the YOLOv8 model, a GPU-enabled machine or cloud instance with at least one GPU is required to ensure efficient computation. Additionally, a minimum of 10 GB of storage is needed to accommodate the dataset and model artifacts. The model operates using Python 3.11.12 and PyTorch 2.6.0 with CUDA acceleration and is executed on an NVIDIA Tesla T4 GPU with 15,095 MiB of memory. To access and download the dataset from Roboflow, an API key is required, which should be securely stored to maintain data privacy and integrity.

### 6.5.5 Color Classification Module

The Color Classification Module analyzes garment images using Euclidean distance calculations to map dominant colors to a predefined palette. This deterministic processing pipeline provides consistent color identification for the recommendation system.

#### Implementation Overviews :

- **Dataset Loading:**

A CSV file (`colours_rgb_shades(myVersion).csv`) is imported using pandas, containing 647 hexadecimal color values with corresponding color names. The module leverages this data to find matches among the dominant colors.

- **Color Extraction Pipeline:**

The pipeline uses OpenCV and K-Means clustering:

The process begins by using images that have already undergone prediction and classification using the previously implemented code. These images, which already focus on specific objects (such as clothing), are further processed using the GrabCut algorithm to refine the segmentation and isolate the main object more precisely.

The first step involves reading the target image and converting it from BGR (default in OpenCV) to RGB format, which is more suitable for color analysis and processing. Once the image is prepared, the GrabCut algorithm is applied to create a mask. This mask functions as a stencil, isolating the main object (a piece of clothing) from the background. By focusing only on the object, subsequent steps are ensured to analyze only the relevant parts of the image.

After isolating the object, the algorithm groups the remaining pixels (those belonging to the object) using K-Means clustering. This step helps identify clusters of similar colors, allowing the system to detect the dominant colors present in the object. To determine the most significant color, the system counts the number of pixels in each cluster. The cluster with the highest pixel count is identified as the dominant color.

The dominant color, represented in RGB values (red, green, and blue components), is then converted into a hexadecimal (HEX) code. HEX codes are widely used in digital platforms and provide a standardized way of representing colors.

Finally, the system compares the extracted HEX color to a set of predefined colors stored in a CSV file. Using the Euclidean distance formula, which calculates the "distance" between two

points in a 3D space (here, the RGB space), the system identifies the predefined color that is closest to the extracted dominant color. The closest color match is then assigned as the semantic color label for the object. This approach ensures that the dominant color is accurately identified and named, providing meaningful and reliable results for further processing or analysis. For used functions, please refer to Appendix 10.7.

#### 6.5.5.1 Technical Environment

The module operates on standard computing. It is developed using Python 3.10 and integrates several key libraries: OpenCV handles image manipulation, NumPy supports efficient numerical operations, scikit-learn's KMeans algorithm performs clustering for dominant color identification, Matplotlib facilitates visualization, and pandas is used for CSV handling.

## 6.6 Testing, Evaluation and Iterative improvements

### 6.6.1 Testing Strategy

Thorough testing of SmartCloset was conducted through both **functional** and **AI-based evaluation**, focusing on feature performance, user experience, and the quality of outfit suggestions. Testing was done iteratively to detect bugs and improve the application with each version.

During the final phase of testing, eight participants were chosen to test the application and then fill a post-testing survey anonymously.

#### 6.6.1.1 Test Design

##### Performance Indicators

- Application loading time
- AI outfit suggestion accuracy
- Responsiveness when adding/removing clothes
- User satisfaction (Post-test survey)

##### Test Scenarios

- Add and remove clothing items
- Access closet with 20+ items
- Get outfit suggestions and evaluate AI relevance
- Trigger the camera to add new items

#### 6.6.1.2 Test Environment

| Device            | OS         | Network    |
|-------------------|------------|------------|
| Samsung A23 Phone | Android 12 | Wi-Fi & 4G |
| Android Tablet    | Android 12 | Wi-Fi & 4G |

Table 8: Test Environment Specifications

### 6.6.1.3 Testing Tools

- Manual interaction & debugging
- AI model tested with different datasets
- Google Forms for post-test user feedback

### 6.6.2 Questionnaire Results

The testing took place on 13 may 2025, participants were shown a demonstration of the application, then tried the application themselves and answered a Post-Test survey. The participants are:

- Dr Maria Habib – PhD holder and Associate Professor at ESIB, USJ
- Dr Tina Yaacoub – PhD holder and Associate Professor at ESIB, USJ
- Joe Maatouk – 4<sup>th</sup> year Architecture Student
- Chiara Edde – 4<sup>th</sup> year CCE Student
- Hussein Dakroub – 4<sup>th</sup> year CCE Student
- Mohammad Medlej – 4<sup>th</sup> year CCE Student
- Mona Hajj Chehade – 4<sup>th</sup> year CCE Student

| Question                                                | Scores                  | Mean | Std. Dev. |
|---------------------------------------------------------|-------------------------|------|-----------|
| How would you rate your overall experience?             | 10, 9, 9, 10, 8, 10, 10 | 9.43 | 0.79      |
| Intuitive and easy to use                               | 5, 4, 5, 5, 4, 5, 5     | 4.71 | 0.49      |
| Visually appealing                                      | 4, 4, 5, 5, 3, 5, 4     | 4.29 | 0.76      |
| Fast and responsive                                     | 5, 4, 4, 5, 4, 5, 5     | 4.57 | 0.49      |
| Struggle to decide what to wear (1 = none ... 5 = high) | 3, 2, 4, 5, 4, 2, 3     | 3.29 | 1.03      |

Table 9: Overall Satisfaction & Usability Scores

How would you rate your overall experience with the app?

8 réponses

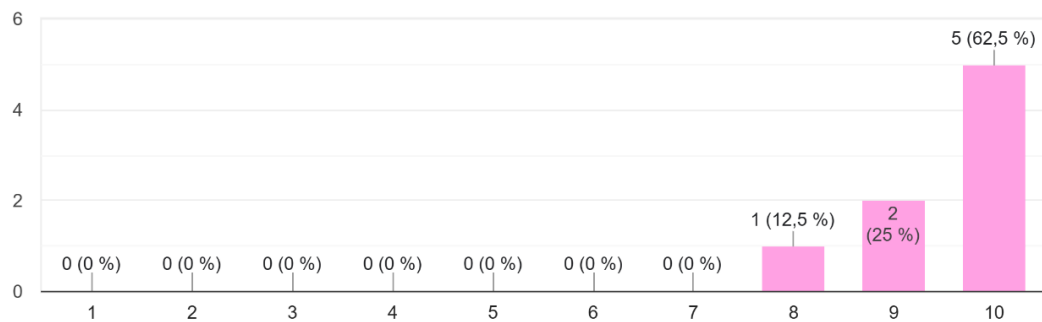


Figure 12: User Experience Rating Distribution

You found the app fast and responsive

8 réponses

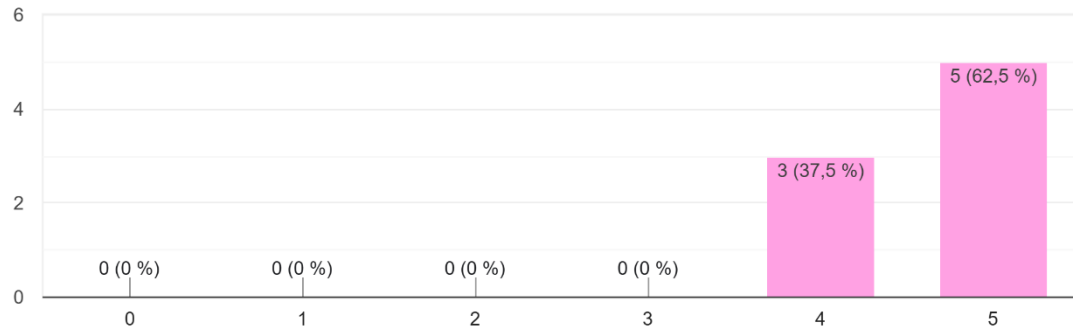


Figure 13:Response Time Satisfaction Distribution

Were you able to understand how to:

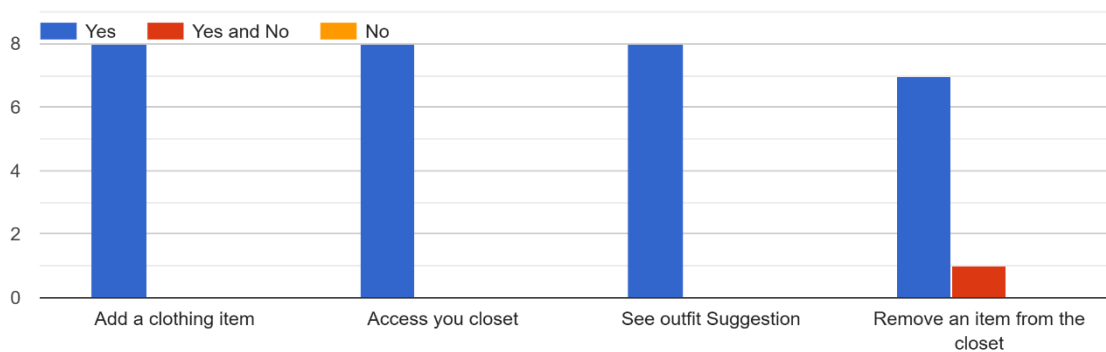


Figure 14:Feature Usability Comprehension Metrics

#### 6.6.2.1 Feature Usefulness

- **Most useful:** outfit suggestions (7 mentions)
- **Unused/confusing features:** none reported

#### 6.6.2.2 Qualitative Feedback Highlights

- Desire to browse friends' closets for inspiration
- Automatic "check-out" of worn items
- Add accessories section

#### 6.6.3 Functional Testing

All features were tested together in a complete flow:

- Add clothing item
- Remove item from closet
- View closet
- Generate outfit suggestion
- Edge cases such as deleting all items

### 6.6.3.1 Bug Detected:

#### Bug 1:

To remove an item from the closet, the camera has to take a picture of the item and run an algorithm of image similarity to update the inventory.

In order to achieve this functionality, the first approach was to always run the camera in the background for it to take picture every one second and run the similarity algorithm to identify if an item is being removed.

This approach caused the database to overload with pictures being take, and the server was receiving requests every one second and had to run a complex algorithm on each picture even if it did not contain any clothing image.

#### Bug 2:

The ultrasonic sensor measured the distance separating the back of the closet and the door to detect if the closet is opened or closed.

When the distance is superior to 50 cm, the Arduino sent a continuous “Alert” message to the application. On each alert, the camera was launched, this caused infinite requests for the camera to be opened as long as the door is open.

### 6.6.3.2 Resolving Methodology:

#### Bug 1:

The problematic behavior was modified as the following:

The camera launches automatically when the user opens the closet, but require manual confirmation to take the photo. This reduced unnecessary uploads and improved database and server performance.

#### Bug 2:

The bug of infinite alerts was solved by sending only one “Alert” message to the application through Bluetooth, and modified the workflow by letting the user take as many pictures as he/she wants, before clothing the remove item screen.

### 6.6.4 Iterative Improvements

| Version | Improvement Description                                                                         |
|---------|-------------------------------------------------------------------------------------------------|
| v1.0    | Initial test with basic dataset with the camera automatically capturing images every one second |
| v1.1    | Implemented Ultrasonic sensor and Changed camera behavior to require manual capture             |
| v1.2    | Modified the “Alert” Arduino behaviour on the door opening                                      |
| v1.3    | Replaced AI training dataset with larger, more diverse set                                      |
| v1.4    | Planned: Personalized model based on user preferences                                           |

Table 10:Version History: Iterative Development and Feature Enhancements

## 7 Impact of the proposed solution

The Smart Closet project is expected to have a multidimensional impact across global, economic, environmental, and societal contexts. This section outlines the potential positive and negative outcomes, both locally and internationally.

### 7.1 Global and Local Impact

**Global Impact:** Smart Closet is aligned with global sustainability trends in fashion, promoting conscious consumption and less textile waste.

With the usage of SmartCloset, the user's closet inventory is easily accessed, reducing the chance of a user to forget about a piece of clothing and buy another similar item.

The statistic section in the SmartCloset application also gives insight for the user about the forgotten clothes (items not worn for more than a month), encouraging the user to donate or recycle those pieces of clothing.

**Local impact:** As mentioned previously, SmartCloset can encourage donating the forgotten clothes of a user, making a difference in the lives of less privileged Lebanese people.

## 7.2 Economic Impact

### Pros:

- Encourages saving by maximizing the usage of existing clothing items.
- Reduces the overconsumption trend of some user

### Cons:

- Initial deployment costs (camera integration, development) may be high for low-income users.

## 7.3 Environmental Impact

### Pros:

- Reduces fashion related waste.
- Decreases environmental footprint associated with overbuying and underusing clothes.
- Contributes to achieving the United Nations Sustainable Development Goal 12 (SDG) by promoting responsible Consumption and Production.

### Cons:

- AI model training, for tasks such as image recognition and outfit recommendation, require computational resources that consume significant energy, contributing to CO<sub>2</sub> emissions.

## 7.4 Impacted Sectors and Communities

- **Sectors:** Fashion retail, AI and software development, second-hand clothing industry, sustainability NGOs.
- **Communities:** Youth, environmentally conscious consumers, and in general everyone.

# 8 Conclusion

## 8.1 Key Milestones:

The Smart Closet system architecture has successfully established seamless dataflow and interaction between the mobile application, the AI model, and the Supabase-backend PostgreSQL database. A robust relational schema was implemented to support core functionalities, and initial training of the YOLOv8 clothing recognition model has laid the groundwork for intelligent outfit suggestions.

The mobile application delivers key features clothing management, closet organization, outfit history, statistics, and outfit recommendations with largely positive usability feedback. Testers confirmed clear understanding of core tasks such as adding/removing clothing items and accessing the closet, through some noted occasional uncertainty with outfit suggestions, indicating an area for refinement. Experience ratings averaged 7–8/10, reflecting the usability with opportunities to enhance intuitiveness. This validates the solution's impact in delivering wardrobe efficiency, personalized styling, and sustainable fashion habits, while iterative improvements will further optimize the user experience.



## 8.2 Future Work:

The next phase of the SmartCloset project will focus on several enhancements that aim to improve functionality, performance, and User experience.

One of the primary goals is to optimize the AI model to enhance recognition accuracy, and expand its capabilities by adding more clothing labels such as hoodie, blouse, shoes, and others. Improve suggestion algorithms, in addition to optimizing outfit suggestion by learning from the user's preferences.

Additionally, new application features will be developed, including a weather tracker for more accurate outfit suggestion, and integration with the user's calendar to suggest outfits according to the event or occasion.

Another key focus will be developing a fully independent backend. While Supabase has been useful for development and testing, we aim for the application to be publicly used, which means a better solution would be to fully develop and deploy the backend and servers. In parallel, the database storage system will be optimized by implementing regular clean up mechanism for deleting clothing images that are no longer mapped on a database record, and delete very old records that are no longer being used, ensuring a more efficient and sustainable system.

Finally, a round of user testing will be conducted to gather feedback on usability, refine the UI/UX, and improve application performance.

These future steps will collectively enhance the performance and user-focused value of SmartCloset, as it evolves toward a publicly deployable solution.

## 9 References

- [1] «Alibaba,» 25 2 2025. [En ligne]. Available: <https://www.alibabagroup.com/en-US/document-1491476795097088000>.
- [2] «Samsung,» 25 2 2025. [En ligne]. Available: <https://news.samsung.com/global/clothing-care-reimagined-1-how-samsungs-innovative-airdresser-is-delivering-comprehensive-clothing-care>.
- [3] «FashionNet,» 25 2 2025. [En ligne]. Available: <https://arxiv.org/abs/1810.02443>.
- [4] «(Samsung Hub),» [En ligne].
- [5] «(digitaltrends-Samsung),» [En ligne].
- [6] «(Samsung-instacart),» [En ligne].
- [7] «(Samsung-Support1),» [En ligne].
- [8] «(Samsung-Support2),» [En ligne].
- [9] «(Smarthings),» [En ligne].
- [10] «(Smarthings-Energy),» [En ligne].
- [11] «(DigitalTrends),» [En ligne].
- [12] «Lucy: The Smart Closet,» 2018. [En ligne]. Available: <https://ami-2018.github.io/Lucy/>.
- [13] Lucy, «Lucy~ The smart closet,» 2018. [En ligne]. Available: <https://youtu.be/NhIsBhbOT8k?si=3a6lgs4Q3fnGy1a->.
- [14] A quick review of every virtual closet app, reddit.com.
- [15] Reviewing the Wardrobe Organising App that Picks Your Outfits, stylewithingrace.com.
- [16] Acloset vs. Stylebook, myindyx.com.
- [17] Create your digital closet, acloset.app.
- [18] «RFID-Wikipedia,» 1 3 2025. [En ligne]. Available: [https://en.wikipedia.org/wiki/Radio-frequency\\_identification](https://en.wikipedia.org/wiki/Radio-frequency_identification).
- [19] «ConflictStrategies,» [En ligne].

## 10 Appendix

### 10.1 Supabase-Kotlin, query to get all available items in a specific closet

```
supabase.from("clothingitem")
    .select(){
        filter{
            eq("closet_id", selectedClosetId)
            eq("availability",1)
        }
    }
```

### 10.2 Supabase Client Initialisation In Kotlin:

```
val supabase = createSupabaseClient(
    supabaseUrl = "https://xxx.supabase.co",
    supabaseKey = "public-anon-key"
)
```

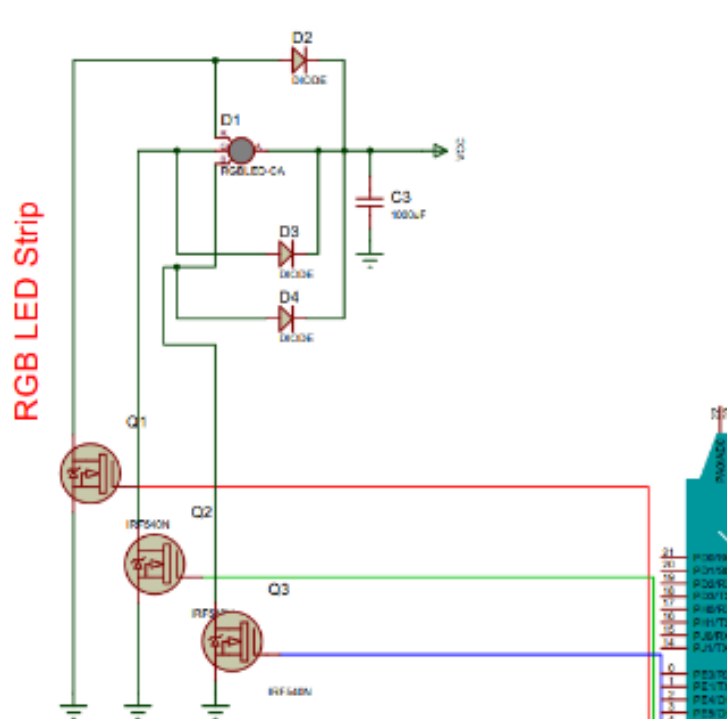
### 10.3 FastAPI Endpoint

```
POST /process-image{
    "clothing_id": "...",
    "image_url": "https://...supabase.co/storage/v1/object/public/..."
}
```

### 10.4 Launching the Server

#### 10.4.1 Uvicorn Local Server

### 10.5 Arduino RGB circuit



## 10.6 Accessing the Main-Fashion Dataset via Roboflow

```
from roboflow import Roboflow
rf = Roboflow(api_key="<API_KEY>")
project = rf.workspace("dataset-yn4f8").project("main-fashion")
version = project.version(5)
dataset = version.download("yolov8")
```

## 10.7 Color Extraction and Analysis Functions:

```
extract_colors(image_path, num_colors=5)
rgb_to_hex(rgb)
display_highest_count_color(colors, counts)
find_closest_hex_color(sorted_colors, color_df)
```